Module: CS2001
Student ID: 190006106
Date: 29/11/2019

Mini Project 2:
Mandelbrot Set Explorer

1 Introduction

The objective of this project is create a GUI-driven Fractal generator that allows users to view and explore the Mandelbrot set graphically.

2 Design

2.1 Packages

1. **main:** contains main application class which will create main window for application.
2. **gui:** contains GUI classes necessary to display user-interface including image view and user input view.
3. **model:** contains Mandelbrot calculator and image creator which will create an image correspond to the Mandelbrot set generated.
4. **property:** Properties class which contains all the default variables value.

2.2 GUIs

My application will have two main components:

- Image view: display the Mandelbrot set and handling mouse event (zoom and pan).
- User input view: display all the parameters (min coordinate, max coordinate in complex plane and magnified value of image) and take input from user (number of iterations, magnified value, colour changing).

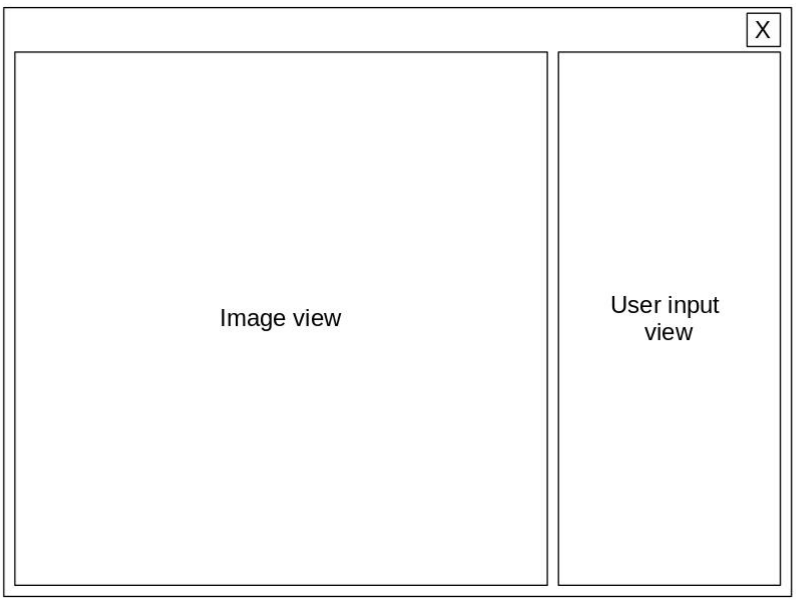


Figure 1: GUI view

2.3 Model

I modified the provided source code for Mandelbrot calculator in order to implement colour mapping functionality. The additional feature in *MandelbrotCalculator* class is recording frequency of each number of iteration appear in the current area except for *maxIterations*. Hence, distribute colour code (0 to 255) to each of the number of iterations depend on the frequency.

In the application, there are two versions of Mandelbrot set which are saved simultaneously.

- **Previous Mandelbrot set:** The image for this version is already calculated and saved. The image view window will use the image of this version to draw.
- **Current Mandelbrot set:** this set is currently generated. When this set is completed, an image for this set will then be constructed and saved as Previous Mandelbrot set.

By saving two versions of Mandelbrot, the operations (zoom, pan, etc) will be executed smoothly on the **Previous Mandelbrot set** since the image view need not to wait for the new image to finish constructing. After **Current Mandelbrot set** generating process finish, the display will be updated. The disadvantage is during the generating process of the new image, the display of Mandelbrot set will have low quality or missing part.

2.4 About requirements

1. **Pan:** I notice that the change of the Mandelbrot set is significant for a small change in value which required user to know the location they want to arrive to pan correctly. Therefore, instead of providing direction and distance, the application will allow use to pan using right click mouse and with the support of super-imposed image, the user can preview new image.
2. **Zoom by drag:** Zoom with rectangle shape will required the application window to resized, otherwise, the new image will have low quality. Therefore, I have decided to restrict the zoom area to be square shape.
3. **Zoom by scroll:** In stead of zoom the image with origin (0, 0) as the pivot, which will make the implementation trivial, I decided to use current position of the mouse pointer as a pivot for the area surround it to be shrink toward or expanded away.
4. **Different colour mappings:** My first approach is distribute colour value (0 to 255) equally for all number of iterations. The result is in Figure 2. Since number of iterations is not equally distributed, when the area is large, most of the pixels have low number of iterations, therefore, dark colour occupy most of the area. Instead of map colour with number of iterations, colour are map depending on the frequency that a particular iterations is appeared.

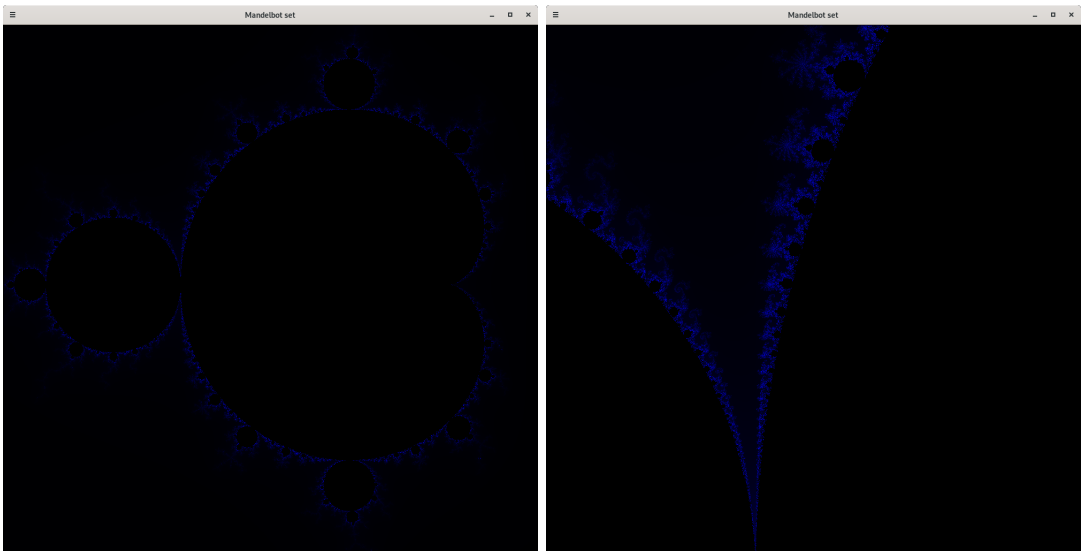


Figure 2: Colour mapping attempts

5. **Undo/Redo:** Undo function will restore the Mandelbrot set using the previous parameters. Redo function will be second undo function which restore the changes from undo function rather than from parameters.

3 Implementation

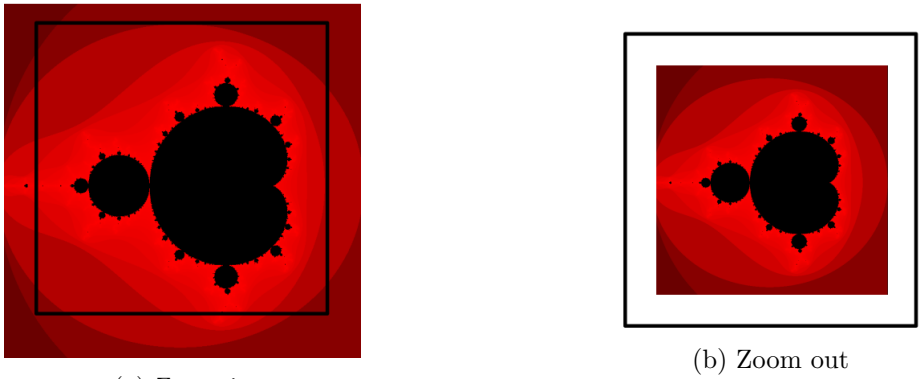
The Mandelbrot calculator generate Mandelbrot set as an 2D array with increasing real and imaginary number from index (0, 0). Therefore, when using this array to create buffered image, which start at top left corner. The image of Mandelbrot was reflected along the real axis (the imaginary number increase downward).

3.1 Display image

3.1.1 Overview

As mentioned, there are two version of the Mandelbrot set. The previous image will always be used to display to image view, when current image completed, previous and current image is the same and, therefore, fill the image view panel exactly. However, when the current version of Mandelbrot image is still generating, previous version of the image may or may not match and occupy all the area in image view. For example, the box with black border indicate the image view:

1. Zoom in: The current Mandelbrot image, which is generating, have smaller area then the previous image, therefore the previous image need to be magnified. (Firgure 3a)
2. Zoom out: The current Mandelbrot image have bigger area then the previous image, therefore the previous image need to be compressed and only occupy part of the image view panel. (Firgure 3b)



(a) Zoom in

(b) Zoom out

Figure 3: Display previous image when processing current image

3.1.2 Underline mechanism

In Figure 4, example of drawing previous image (small image) at correct position to the current image (which will occupy all the panel represent with black border).
Find the size of previous image relating to current image with this formula

$$previousSize = scale * currentSize = \frac{previousRange}{currentRange} * currentSize$$

Where, in this example (Figure 4):

- Current size = 1000 (pixels)
- Previous range = 3.0
- Current range = 6.0

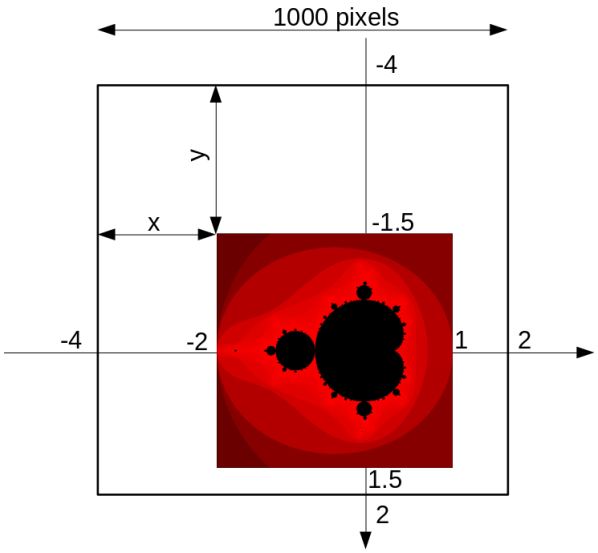


Figure 4: Example of zoom out

Therefore, previous size = 500 (pixels)
Find coordinate (x, y) to draw from $(0, 0)$, top left of panel.

$$x = \frac{previousMinReal - currentMinReal}{currentRange} * currentSize$$
$$y = \frac{previousMinImaginary - currentMinImaginary}{currentRange} * currentSize$$

Where, in this example (Figure 4):

- Current size = 1000 (pixels)
- Current range = 6.0
- previous min real = -2.0
- current min real = -4.0
- previous min imaginary = -1.5
- current min imaginary = -4.0

Therefore, x = 333 (pixels) and y = 417 (pixels)
(See *ImageView* class line 139 - 150)

3.1.3 Super-imposed image

Using BufferedImage of type TYPE_3BYTE_BGR is enough to represent the image of Mandelbrot set and super-impose image when panning and zooming, however, I decided to use TYPE_4BYTE_ABGR which have additional 1 byte for alpha value. By setting alpha to a low value, the super-imposed image will not hide the actual image of the Mandelbrot set. Therefore, with one image of Mandelbrot set, I can draw the actual image and the preview (super-imposed image) of panning and zooming by changing the size and location of the second image. (Figure 5)

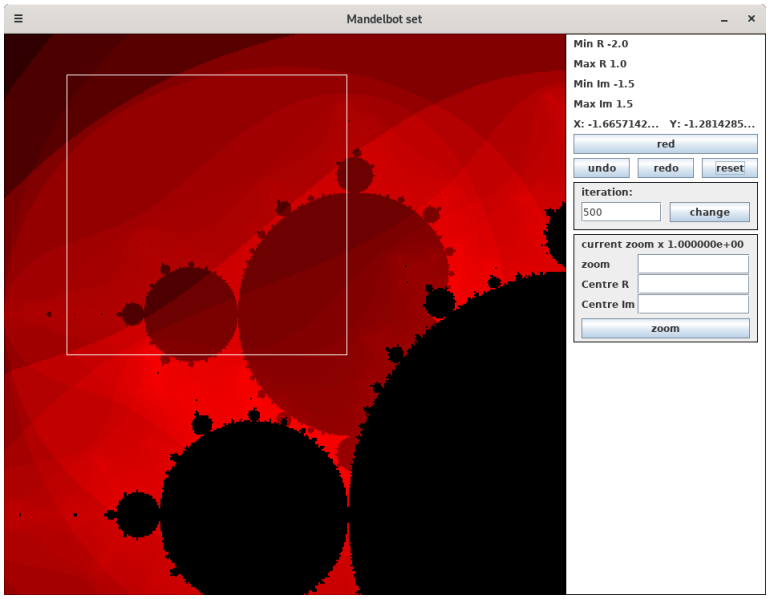


Figure 5: Example super imposed image (on top of the actual image) while zooming

3.2 Requirement

3.2.1 Pan and zoom by drag

At first, I treat panning and zooming as separate functionalities and provide a switch in user input field for user to change between panning and zooming with left mouse button. However, it is possible to use left mouse button for zooming and right mouse button for panning. Nonetheless, a flag is needed to specify which function is being used so two functionalities will not interfere with each other.

3.2.2 Zoom with magnified value

To zoom image with the input magnified value, instead of one input (zoom value), I also provide options to indicate the centre location as the pivot for zooming in. If the input is left blank, the program will use the current centre coordinate as the pivot. Furthermore, I also add an extra feature which records the last coordinate mouse pressed (left-click) event on the image and saves it in the input text box. (Figure 6)

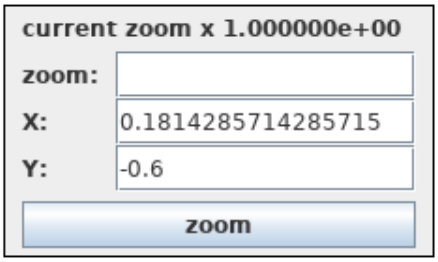


Figure 6: Cursor position saved, user only need to provide zoom value

3.2.3 Alternate Iterations

Large values for iterations will cause the application to take too long to generate the Mandelbrot set. Therefore, I have provided a restriction which will make sure the program only runs if the input is not out of bound. The restriction range can be modified in *Properties* class.

3.2.4 Undo and Redo

For the implementation of undo and redo, I used two stack objects to retrieve the most recent parameters at the top.

- Undo stack: This stack will store all the previous parameters from previous images before they are updated to the current image.
- Redo stack: This stack will store all the previous parameters that the undo function used.

When using undo function, parameters from images will be pushed to the redo stack and parameters at the top of the undo stack will be used as new parameters for the current image. if parameters from image push to undo stack when using the redo function, the records can be significantly large since I implemented **zoom with scroll**. Therefore, I decided to implement the redo function to restore changes from the recently executed undo functions and the redo stack will be cleared when a pan/zoom is operated.

3.2.5 Different colour mappings

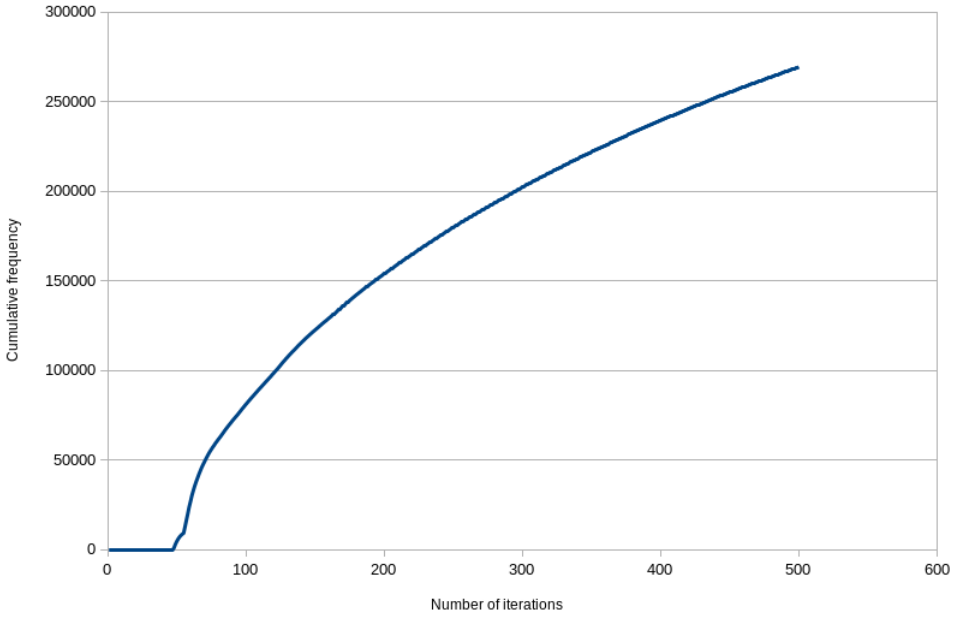


Figure 7: Iterations value and cumulative frequency

The data in the graph is collected from a Mandelbrot set with $maxIteration = 500$.
To distribute the colour values, I calculate the cumulative frequency using the frequency for each value of iterations from 0 to $maxIteration - 1$ represented in Figure 7. Next step is to map the cumulative frequency to the colour values as shown in Figure 8.
From Figure 8, values of iteration from 0 to around 50 have low frequencies so they all have colour value 0. Meanwhile, values of iteration from 50 to around 70 have very high frequencies so each of them has a distinct colour value. If a value of iteration has a high frequency and occupies more than one colour values, the mid-valued colour is chosen.
(See *MandelbrotCalculator* class line 123 - 146)

3.2.6 Zoom with scroll

First step is finding the coordinate $(splitX, splitY)$ of the cursor on the complex plane using the following formula.

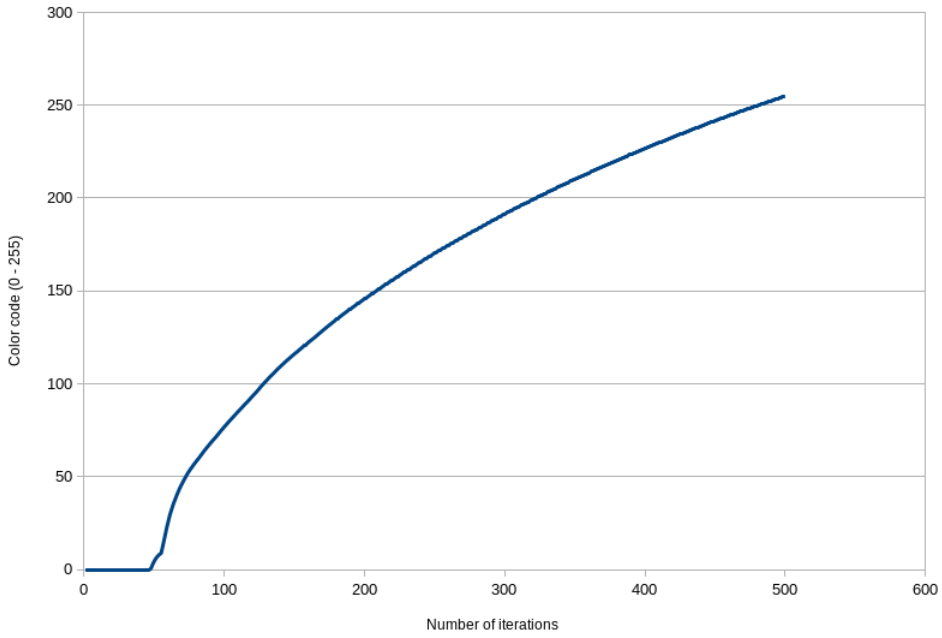


Figure 8: Iterations value and colour value (0 - 255)

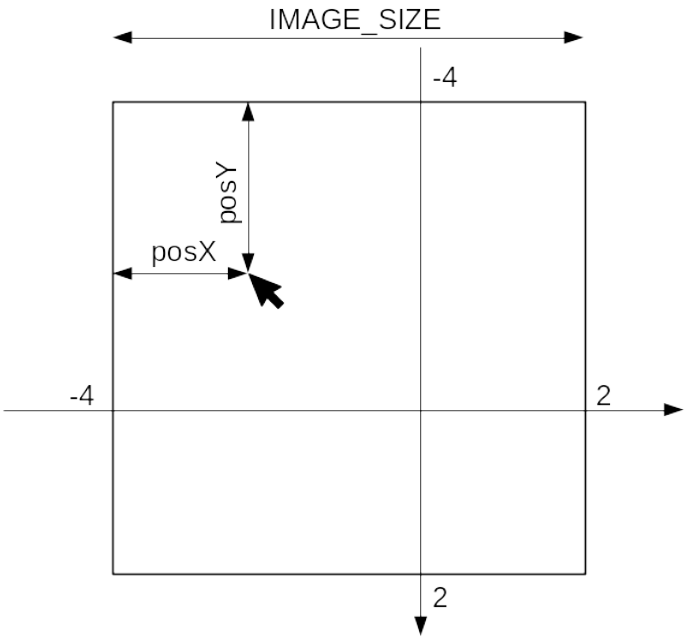


Figure 9: Example of split coordinate

$$splitX = range * \frac{posX}{IMAGE\_SIZE} + minReal$$
$$splitY = range * \frac{posY}{IMAGE\_SIZE} + minImaginary$$

Where, in this example (Figure 9):

- range = 6.0
- posX = 300 (pixels)
- posY = 400 (pixels)

- IMAGE_SIZE = 1000 (pixels)
- minReal = -4.0
- minImaginary = -4.0

Therefore, $splitX = -2.2$ and $splitY = -1.6$.
With split coordinate, calculate new min real, min imaginary by expand or shrink the distance between min value and split value. calculate new range by expand or shrink the range itself.

$$\begin{aligned} range &= range * zoomFactor \\ minR &= splitX - (splitX - minR) * zoomFactor \\ minI &= splitY - (splitY - minI) * zoomFactor \end{aligned}$$

Where, in this example (Figure 9):

- splitX = -2.2
- splitY = -1.6
- range = 6.0
- minReal = -4.0
- minImaginary = -4.0
- zoomFactor = 0.95 (zoom in) = 1.05 (zoom out)

Therefore, when $zoomDirection = 1$, $range = 6.3$, $minR = -4.09$, $minI = -4.12$. Hence, the split coordinate will be stationary respect to the image view panel when scrolling.

3.3 UML diagram

3.3.1 main package

MainApplication
+ main(String[])

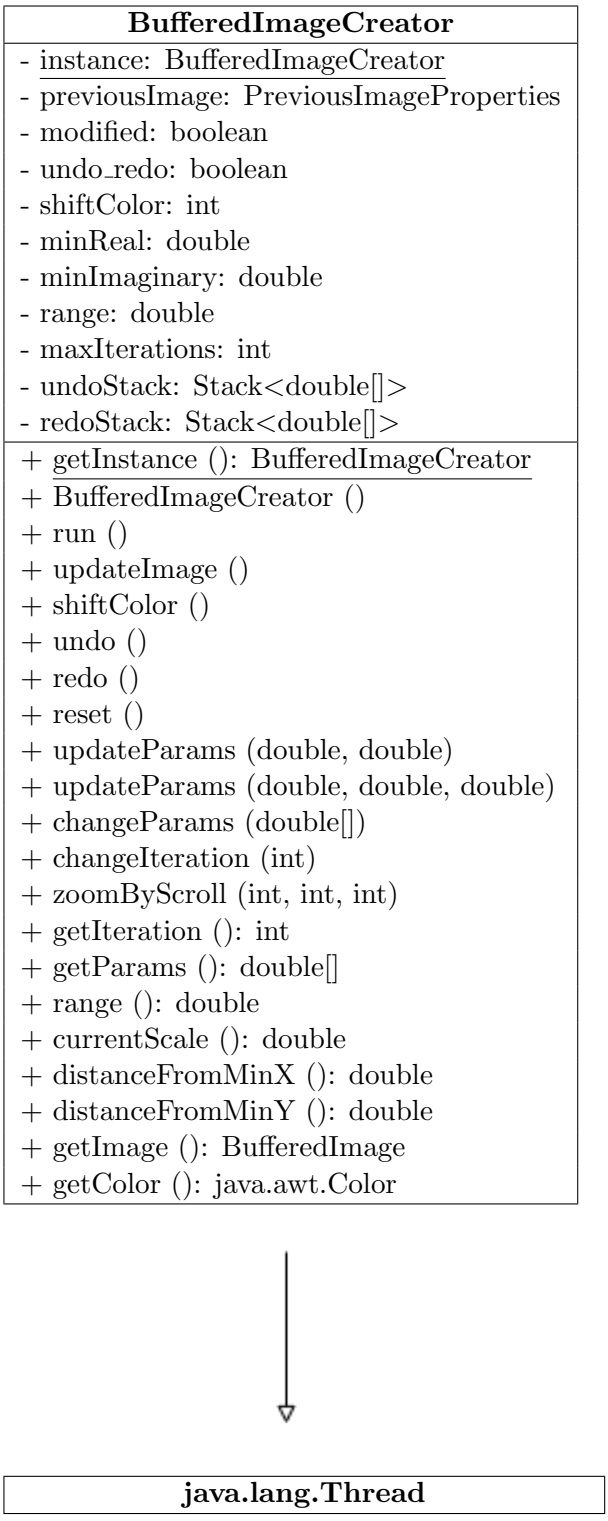
3.3.2 property package

Properties
+ DEFAULT_MIN_REAL: double
+ DEFAULT_MIN_IMAGINARY: double
+ DEFAULT_RANGE: double
+ DEFAULT_RADIUS_SQUARE: double
+ DEFAULT_SCROLL_FACTOR: double
+ DEFAULT_ITERATIONS: int
+ DEFAULT_MIN_ITERATIONS: int
+ DEFAULT_MAX_ITERATIONS: int
+ IMAGE_SIZE: int
+ UPDATE_IMAGE_WAIT_TIME: int
+ REPAINT_WAIT_TIME: int
+ IMAGE_ALPHA_VALUE: int

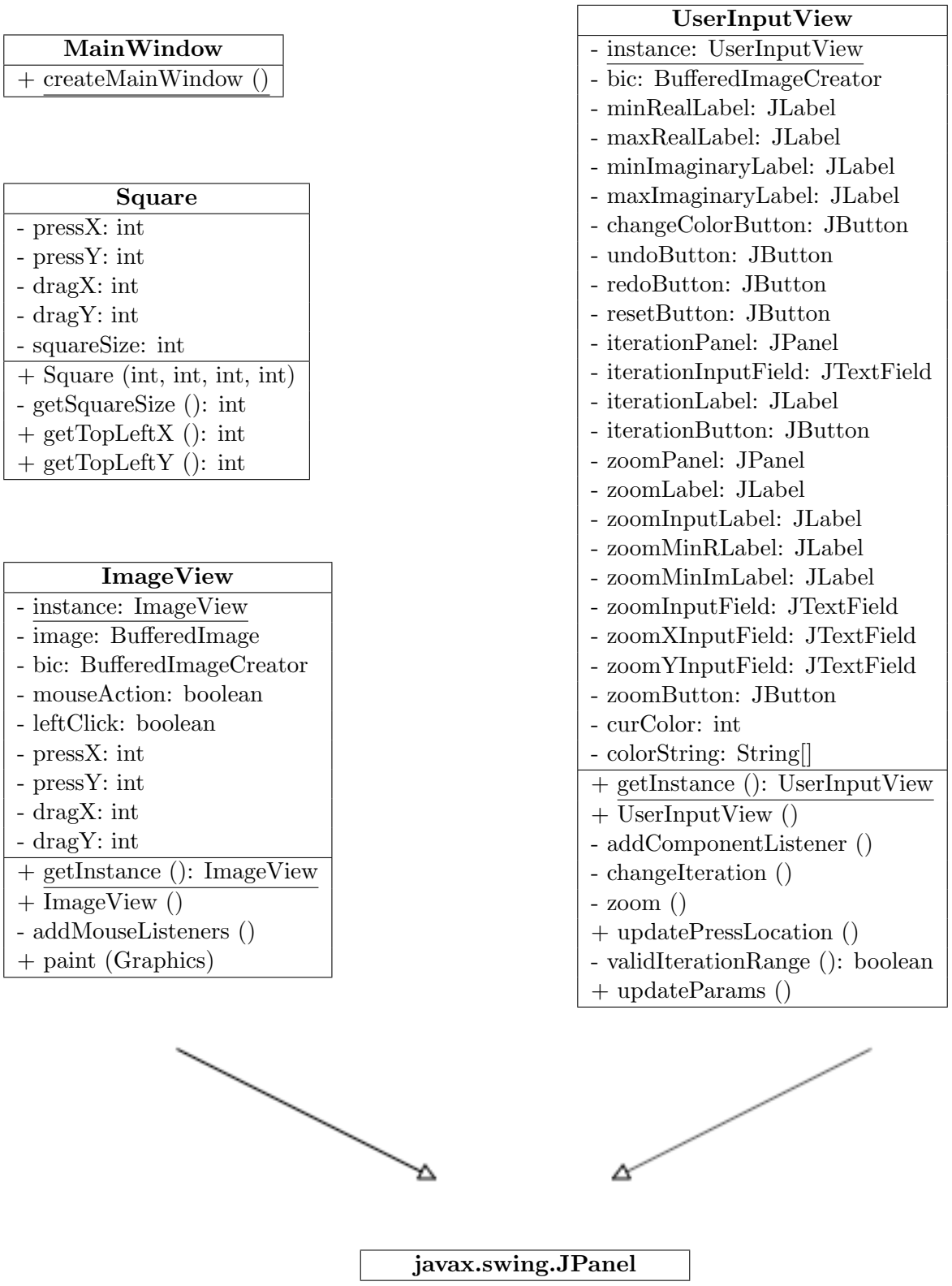
3.3.3 model

PreviousImageProperties
- image: BufferedImage - previousMinR: double - previousMinI: double - previousRange: double - previousColor: java.awt.Color
+ setBoundary (double, double, double) + setImage (BufferedImage, java.awt.Color) + getColor (): java.awt.Color + getImage (): BufferedImage + getPreviousParams (): double[]

MandelbrotCalculator
- colorCode: int[]
+ calcMandel (double, double, int, int): int + calcMandelbrotSet (int, int, double, double, double, double, int, double): int[][] + getColorCode (): int[]


BufferedImageCreator
- instance: BufferedImageCreator
- previousImage: PreviousImageProperties
- modified: boolean
- undo_redo: boolean
- shiftColor: int
- minReal: double
- minImaginary: double
- range: double
- maxIterations: int
- undoStack: Stack<double[]>
- redoStack: Stack<double[]>
+ getInstance (): BufferedImageCreator
+ BufferedImageCreator ()
+ run ()
+ updateImage ()
+ shiftColor ()
+ undo ()
+ redo ()
+ reset ()
+ updateParams (double, double)
+ updateParams (double, double, double)
+ changeParams (double[])
+ changeIteration (int)
+ zoomByScroll (int, int, int)
+ getIteration (): int
+ getParams (): double[]
+ range (): double
+ currentScale (): double
+ distanceFromMinX (): double
+ distanceFromMinY (): double
+ getImage (): BufferedImage
+ getColor (): java.awt.Color
java.lang.Thread

3.3.4 gui


MainWindow
+ createMainWindow ()
Square
- pressX: int
- pressY: int
- dragX: int
- dragY: int
- squareSize: int
+ Square (int, int, int, int)
- getSquareSize (): int
+ getTopLeftX (): int
+ getTopLeftY (): int
ImageView
- instance: ImageView
- image: BufferedImage
- bic: BufferedImageCreator
- mouseAction: boolean
- leftClick: boolean
- pressX: int
- pressY: int
- dragX: int
- dragY: int
+ getInstance (): ImageView
+ ImageView ()
- addMouseListeners ()
+ paint (Graphics)
UserInputView
- instance: UserInputView
- bic: BufferedImageCreator
- minRealLabel: JLabel
- maxRealLabel: JLabel
- minImaginaryLabel: JLabel
- maxImaginaryLabel: JLabel
- changeColorButton: JButton
- undoButton: JButton
- redoButton: JButton
- resetButton: JButton
- iterationPanel: JPanel
- iterationInputField: JTextField
- iterationLabel: JLabel
- iterationButton: JButton
- zoomPanel: JPanel
- zoomLabel: JLabel
- zoomInputLabel: JLabel
- zoomMinRLabel: JLabel
- zoomMinImLabel: JLabel
- zoomInputField: JTextField
- zoomXInputField: JTextField
- zoomYInputField: JTextField
- zoomButton: JButton
- curColor: int
- colorString: String[]
+ getInstance (): UserInputView
+ UserInputView ()
- addComponentListener ()
- changeIteration ()
- zoom ()
+ updatePressLocation ()
- validIterationRange (): boolean
+ updateParams ()
javax.swing.JPanel

4 Testing

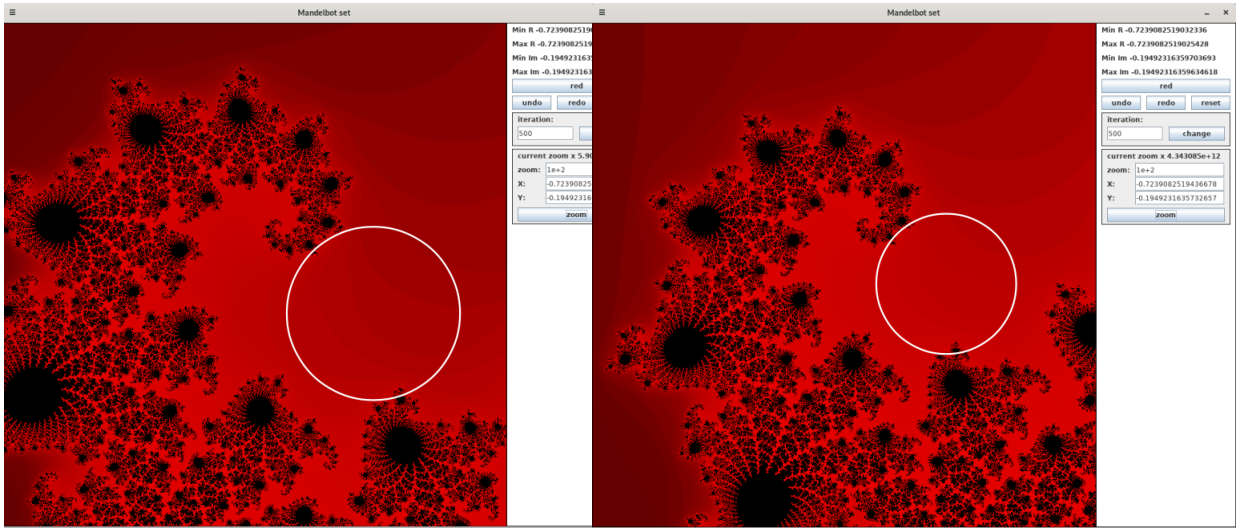
Test no	Test purpose	Execution	Expected result	Conclusion
1	Pan functionality	Right-click and drag to pan the image	New image is shifted towards the dragging direction	Success
2	Zoom by drag functionality	Left-click and drag to zoom	A best fit square with white border appears showing the area under magnification. A super-imposed image, which shows the overview of the magnified area, is clear on top of the actual image, which can be seen under the super-imposed image.	Success
3	Ensure Pan and zoom by drag functionalities not interfere each other	Right-click and drag (to use pan functionality), left-click before releasing the right mouse button. Similarly, right click after zooming	No change is made since pan and zoom cancel each other	Success
4	Zoom with magnified value without centre coordinate	In zoom panel, input the zoom value and leave the X and Y field empty.	Image will scale with centre coordinate stationary	Success
5	Zoom with magnified value with centre coordinate	In zoom panel, provided zoom value and centre coordinate	Image will scale with centre coordinate specified and hit 'enter' or press zoom button (i.e. zoom value 1 and coordinate (-0.5, 0) will reset to default image)	Success
6	Use save coordinate functionality	Left clicked to desired point on image then specify zoom value to perform zoom with the point as centre coordinate.	A coordinate specify the coordinate of the clicked point in 2D complex plane is save to input text field (X and Y). With specified zoom value, the image will scale around the centre coordinate.	Success
7	Invalid input for zoom value, centre coordinate.	Provide string not match double format or non-positive value for zoom value.	All input text field is reset and no change is made	Success
8	Alternate max iteration to change precision.	In Iteration panel, provide valid iteration and hit 'enter' or press change button	Image will automatically update to give more precision if provide higher max iteration number and vice versa.	Success

Test no	Test purpose	Execution	Expected result	Conclusion
9	Invalid input for max iterations	Provide string not match integer format or out of range for iterations which is specified in *Properties* class.	The iterations number will be reset to the current max iterations	Success
10	Undo functionality	Perform several changes to Mandelbrot set and then undo.	Changes being undo and eventually come back to the original image.	Success
11	Redo functionality	Perform several changes to Mandelbrot set then some undo operation and redo.	After undo, redo will again change the image similar to the changes previously been made to the Mandelbrot set and eventually return to the last changes made to Mandelbrot.	Success
12	Clear redo stack	Similar to test 11, however, before redo, made a changes to Mandelbrot set (which will clear redo stack)	No change to image when redo since there are no recent undo operations being made.	Success
13	Colour mappings	Zoom by any functionalities or pan.	Even though the Mandelbrot set changes and the number of iterations value for each pixel is changed, the range of colour value from 0 to 255 is used because a colour value is not assign to a value of iteration permanently but change depending on the frequency.	Success (See Figure 10
14	Change colour theme	press colour button label red or blue or green.	colour theme of image will be changed similar to the label	Success
15	zoom with scroll	Position mouse cursor inside image view panel at pivot coordinate to zoom and scroll to zoom image.	The image will shrinks or expands around the cursor pointer, coordinate point by cursor will be stationary with respect to the panel.	Success
16	Zoom by scroll not interfere with pan or zoom by drag functionalities.	Perform pan and scroll by drag functionality, before release mouse button to execute the request, scroll to zoom.	Scroll will not zoom the image and have no effect on the Mandelbrot set while pan and zoom by drag	Success
17	Reset image to default	Press reset button	The Mandelbrot set parameters will be reset to default value specified in *Properties* class.	Success

Test 13

Figure 10 show the an area of Mandelbrot set with Figure 10a zoom further in than Figure 10b. Area

in circle show that for the same area, when changing the scale of Mandelbrot, the frequency of value of iteration is changed, hence, change the colour value map to it previously.



(a) Mandelbrot set 1 (b) Mandelbrot set 2

Figure 10: Colour Mapping changes

5 Conclusion

In term of completeness, I have attempted all the basic requirements and three out of four extensions. I have spent a significantly more amount of time for **Different Colour Mappings** and **Zoom animations** than for **Full Undo, Redo, Reset**. I wanted to focus on two extensions in order to make the implementation as good as possible. However, since undo and redo is also one of the basic requirements, I decided to attempt Full Undo, Redo, Reset extension which I found very helpful in the testing process.

Java Swings have a lot of functionalities which I have not been able to learn yet such as functions in the **BufferedImage** or **Graphics2D.setRenderingHint**. While reading about **BufferedImage** class in Oracle, I came across several graphics concept which, in my opinion, has prevented me to use some of its functionalities. Using different rendering hint to prioritise quality/time efficiency might also have a huge impact on my application performance.

Reference

Reading list:

- Performing Custom Painting (https://docs.oracle.com/javase/tutorial/uiswing/painting/index.html)
- BufferedImage class (https://docs.oracle.com/javase/7/docs/api/java/awt/image/BufferedImage.html)
- Controlling Rendering Quality (https://docs.oracle.com/javase/tutorial/2d/advanced/quality.html)
- KeyListener (https://docs.oracle.com/javase/7/docs/api/java/awt/event/KeyListener.html)
- Timer (https://docs.oracle.com/javase/7/docs/api/javax/swing/Timer.html)
- Layout Managers (https://docs.oracle.com/javase/tutorial/uiswing/layout/visual.html)

Source code:

- For drawing BufferedImage I modified code from https://possiblelossofprecision.net/?p=526.
- MouseEvents (CS2101_S10-12_GUIs lecture slide 31,32).